

Benchmarking suite

Anonymous Author(s)

Abstract

A clear and well-documented $\text{\LaTeX}$ document is presented as an article formatted for publication by ACM in a conference proceedings or journal publication. Based on the “acmart” document class, this article presents and explains many of the common variations, as well as many of the formatting elements an author may use in the preparation of the documentation of their work.

1 Introduction

The rapid growth of machine learning across the computing spectrum, from resource-constrained edge devices to high-performance data center nodes, has necessitated a rigorous focus on system efficiency and performance characterization. As models scale in size and deployment environments become increasingly heterogeneous, the ability to benchmark performance accurately is crucial not just for comparison, but also for system co-design. However, we identify multiple shortcomings of existing standardized benchmarks when used by researchers as a tool for systematic performance analysis of modern machine learning systems.

First, the architectural assumption of linear execution is breaking down. Traditional benchmarks typically operate on static, acyclic computation graphs, modeling systems as a simple feed-forward pipeline. This abstraction fails to capture the dynamic control flow inherent in emerging agentic workflows and Retrieval-Augmented Generation (RAG) systems, which introduce input-dependent control flow, cycles, and dynamic branching.

Second, existing benchmarking suites are typically segmented by scale (e.g., separating *Tiny* from *Edge* tracks), creating significant coverage gaps for hardware that does not fit neatly into these rigid categories. This segmentation often leads to the under-utilization of mid-range hardware.

Third, standard tools often report a skewed picture of efficiency by focusing solely on accelerator throughput. This disregard for end-to-end data movement, including data loading and preprocessing, masks the I/O and CPU bottlenecks that frequently dominate real-world performance, particularly in computer vision and multimodal pipelines. Simultaneously, existing benchmarks largely operate in isolation, failing to account for workload collocation. In production, particularly at the edge, where resources are scarce, workloads must contend for shared resources. The lack of collocation-aware instrumentation means that interference patterns remain invisible to system architects.

To address these gaps, we introduce *McBenchface*, a modular framework designed for rigorous end-to-end performance analysis of machine learning systems at a single-node scale.

Our solution begins by decomposing the logical execution into independent stages. This architecture naturally supports arbitrary non-linear execution graphs, including the cyclic dependencies and dynamic fanouts characteristic of agentic loops and RAG workflows. Beyond modeling complexity, this modularity allows researchers to perform precise ablation studies by swapping individual components, such as replacing a vector database while keeping the generation model constant, to isolate specific performance bottlenecks.

We assemble these stages into pipelines using a declarative configuration system. This configuration system not only controls the execution graph of the pipeline, but can also inject configuration to the specific stages, such as swapping the model used in the inference stage. This design naturally eliminates the gaps caused by segmenting benchmarks by hardware scales and allows for no-code scalability studies, where users can easily verify the performance of their system under different conditions.

The framework expands its execution scope to account for real end-to-end execution of modern systems. By doing so, the framework exposes critical data movement bottlenecks, such as preprocessing latency and storage I/O, which are often masked by benchmarks that preload preprocessed data. In addition to latency and throughput, we also treat profiling as a core component through deep integration with RadT, a framework for data management and visualization of hardware-aware machine learning experiments. Furthermore, we take advantage of the RadT’s collocation-aware profiling information to allow for interference studies, giving users the possibility of launching an arbitrary number of concurrently running pipelines.

Contributions:

- We characterize the limitations of standardized benchmarks as tools for exploratory ML systems research, identifying specific pain points related to flexibility, end-to-end analysis, scalability, and collocation.
- We present the design and implementation of *McBenchface*, a novel, compositional framework for ML systems analysis that enables rapid experimentation with complex and dynamic workload structures.
- We introduce a unified profiling methodology that correlates high-level application metrics with low-level hardware metrics, providing visibility over bottlenecks and resource contention in studied workloads.
- We validate *McBenchface*’s utility through series of case studies, demonstrating its effectiveness for complex tasks such as LLM scalability analysis, RAG pipeline ablation, and workload collocation studies.

2 Background and Related Work

Standardized benchmarks. Standardized benchmarking provides necessary foundation for fair comparison of ML systems performance across diverse architectures. Currently, the landscape is anchored by MLPerf suite of benchmarks, which has established itself as the standard for accelerator characterization. Through its closed division, MLPerf enforces strict guidelines on the use of model architecture and weights to isolate hardware performance. However, the structural choices required to achieve this standardization have created significant overage gaps. MLPerf segments its suite into discrete tracks defined by hardware scale, such as *Tiny* for microcontrollers, *Edge* for edge servers, and *Datacenter* for server clusters. This segmentation creates gaps for the growing classes of hardware. This includes devices such as NVIDIA Jetson and Coral EdgeTPU, which are too powerful for *Tiny* workloads, yet they lack the resource to run *Edge* workloads or these workloads are unrealistic for this scale of hardware. Consequently, researchers are unable to perform continuous scalability studies with a single benchmark, making it difficult to identify the precise architectural breaking points of a system as model sizes increase.

The broader ecosystem remains fragmented. EEMBC ML-Mark targets embedded inference but has seen limited updates, struggling to keep up with the rapid evolution of Transformer-based architectures. AI Benchmark provides extensive coverage of mobile SoCs but functions primarily as a consumer-grade rather than a systems research tool. On the other end of the spectrum, TPCx-AI attempts to offer an alternative by modeling end-to-end pipelines, providing scaling factors to alter the size of benchmark’s workloads. However, its implementation is rooted in enterprise analytics stacks. This imposes high resource requirements even for the minimum scaling factor, which often exceeds the resources of edge devices. Furthermore, this benchmark targets primarily classical machine learning workloads, with modern deep learning workloads representing minority of the benchmark.

End-to-end benchmarking. To isolate the performance of systems, benchmarks like MLPerf often operate on preprocessed data, preloaded in memory. This can create a misleading view of performance that hides the significant latency const of the data loading and preprocessing stages of machine learning pipelines.

Other studies have demonstrated that these stages account for X. By excluding these stages from primary metric, standardized benchmarks incentivize hardware design that maximize peak FLOPs of the accelerator while neglecting the I/O bandwidth and CPU performance required to feed the accelerator.

Linear execution. The prevailing model for machine learning execution is a static, feed-forward execution graph. MLPerf

Inference assumes a linear flow where request enters, is processed by forward pass of deep neural network, and produces an output. As the landscape shifts from static execution to the rising Compound AI Systems (CAIS), agents and thinking transformer-based models, this abstraction no longer captures the full view of the execution of modern ML systems.

Modern workloads exhibit dynamic control flow that breaks static benchmarking assumptions. Agentic loops introduce cyclic dependencies where the output of one iteration determines the input of the next. Similarly, Retrieval-Augmented Generation (RAG) pipelines utilize conditional branching, dynamically deciding whether to retrieve external context and from which source, or answer directly based on the semantics of the query. A benchmark that cannot model cycles and conditional branching fails to capture the real-world performance of this new class of workloads.

Collocation and profiling. The assumption of isolation in benchmarking also diverges from the real-world deployments, especially at the edge where embedded devices and personal computers are increasingly expected to run multiple machine learning workloads concurrently. This might involve collocation of background fine-tuning workload alongside a foreground inference workload. Collocation introduces interference as workloads contend for shared resources.

Standardized benchmarks treat collocation as an orthogonal problem. MLPerf operates primarily in isolation mode to ensure reproducibility, leaving the orchestration of multi-tenant scenarios entirely to the user. While TPCx-AI supports concurrent streams to measure aggregate throughput, it does not provide the instrumentation required to analyze fine-grained interference patterns and, as mentioned earlier, majority of workloads include traditional ML workloads. Configuring complex collocation mechanisms, such as NVIDIA MPS or MIG, requires significant manual effort and expertise. Without a framework that treats collocation-aware profiling as a first-class citizen, researchers lack the visibility needed to correlate application-level performance degradation with hardware-level contention events.

Standardized benchmarks as a tool for ML systems research. Finally, the goals of standardization and the needs of systems research can be at odds for some use cases. To ensure fair comparisons, standardized benchmarks mandate the use of monolithic implementations designed to execute specific model in highly controlled manner. While this rigidity is necessary for comparing hardware, it can be suboptimal for exploratory research and broader performance analysis of ML systems.

Systems researchers typically perform ablation studies, where a single component (e.g., the data loader or retrieval engine) is modified to isolate its impact on system performance, and do so across various kinds and sizes of workloads to ensure its scalability. To achieve this in standardized benchmarks, researchers are required to make invasive

changes to a tightly coupled codebase, making these changes non-transferrable between workloads.

Some closing statement. Here we should say how we complement the existing benchmarks specifically as a tool for ML systems research.

rob *Need to add AISBench which is fairly new but seems to be the closest tool right now. They argue for end-to-end benchmarking. It is still a benchmark, but they claim to provide a more comprehensive view of execution. Similarly to TPCxAI they can do multistream, and similarly to us have different load generation patterns. They focus on larger servers+clusters. The paper should also provide a good related work overview to fill gaps with other benchmarks. Overall they say that there is a demand for this more end-to-end comprehensive benchmarking from industry partners. They claim to have involved 20 partners.*

rob *HPI also has a vldb preprint which looks at tpcxai and cites our tdis vision paper. We should look at what they say about where the field should be moving towards.*

3 The McBenchface framework

McBenchface is a modular experimentation platform designed to address the limitations of rigid benchmarks for ML systems research. As illustrated in Figure 1, it enables researchers to construct, analyze, and experiment with complex ML workloads by modeling them as flexible, instrumented pipelines. The design of *McBenchface* is guided by three core principles: modularity and composition, declarative configuration, and integrated, collocation-aware profiling.

Modularity and Composition. *McBenchface*’s fundamental principle is the decomposition of complex ML workloads into independent reusable components or *stages*. A *stage* can encapsulate any unit of work, from data preprocessing and model inference to custom logic for routing. These *stages* are then composed into a *pipeline*, which is a directed graph that defines the flow of data. This modular, graph-based architecture allows researchers to model complex execution graphs, including fan-out and cyclic patterns shown in Figure 1. This is particularly important for representing modern, dynamic workloads such as agentic systems or complex RAG pipelines, which often feature iterative, self-reflective loops that are currently not covered by traditional, linear benchmarks. Unlike production-oriented computing engines, which prioritize distributed fault-tolerance and high-throughput scaling, *McBenchface*’s design is intentionally minimalistic. This approach avoids the complex overheads of these production systems that can inadvertently mask the very performance bottlenecks and interference patterns that researchers aim to identify and analyze.

Declarative Configuration. To maximize agility, *McBenchface* maintains a strict separation between composition and logic. The pipeline topology, component wiring, and stage

parameters are defined declaratively in YAML configuration files. This allows researchers to reconfigure workloads by swapping stages, modifying parameters, or altering the graph structure, without modifying framework code. *McBenchface* does not attempt to implement complex control flow or algorithmic logic within the configuration files. Instead, this logic is implemented in the *stages*, ensuring the configuration remains a high-level map of the experiment. This design lowers the barrier to performing ablation studies, such as replacing a inference model or retrieval engine, simply by changing a few lines in the configuration file.

Integrated, Collocation-Aware Profiling. As show in Figure 1, *McBenchface* leverages the RadT’s platform, a system specialized in collocation-aware instrumentation and subsequent data management of experimental data, to deliver multi-layered performance analysis out-of-the-box. Each pipeline is executed in a separate process to accurately model the resource contention that occurs in real-world, multi-tenant deployments. A new tracing infrastructure was co-developed to automatically capture high-level application events and correlate them with detailed, system-level hardware metrics. Users can visualize these as in a unified trace across multiple collocated workloads.

3.1 Core Architectural abstractions

McBenchface’s architecture, illustrated in Figure 1, is built on three key abstractions: the *stage*, the *pipeline* and the *load generator*. Together, they provide a flexible and extensible system for defining and executing complex ML workloads.

3.1.1 Stage. The *stage* is the fundamental, reusable building block in *McBenchface*, representing a single logical step in a workload, such as data loading, preprocessing, model inference, output formatting, or conditional semantic routing. To maximize ease of use for researchers, creating a new *stage* is a lightweight process. A new *stage* inherits from a abstract base class and implements a run method for the core logic and a prepare method for one-time setup. This base class handles the parsing of the stage’s parameters from the configuration file and inter-stage communication via queues, abstracting these complexities away from the users, allowing them to focus on the core logic of the stage.

Each *stage* is executed in its own thread, consuming data from its input queues and pushing results to output queues. This approach allows optional request pipelining through the graph, and possible fan-outs in topologies. For stages with multiple inputs, the framework provides configurable queue polling policies which determine how data is consumed, as detailed in Section 3.2.

3.1.2 Pipeline. A *pipeline* defines the structure of a workload as a directed graph of connected *stage* instances. This graph is defined declaratively in the configuration file, where each stage is assigned a unique identifier and output *stages*’

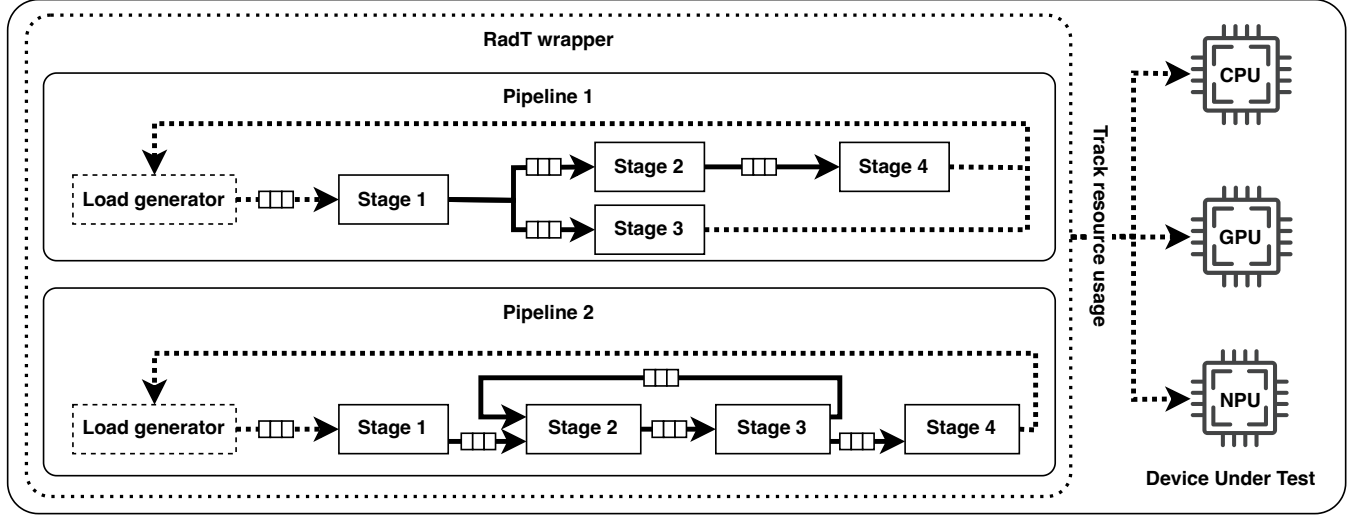


Figure 1. System architecture of the *McBenchface* framework, visualizing two concurrent pipelines with fan-outs and cycles.

identifiers, to which the *stage* will be automatically connected using queues. This declarative, graph-based composition is what gives *McBenchface* its expressive power, allowing researchers to model not only simple linear sequences but also complex topologies like the fan-out in Pipeline 1 and the cycle in Pipeline 2 of Figure 1. To help researchers manage this complexity and verify their designs, *McBenchface* automatically generates a visual representation of the pipeline graph from the configuration files.

3.1.3 Load Generator. The *load generator* drives the pipeline’s execution using a signal-based approach, as indicated by the dotted arrows in Figure 1. It sends structured *query* objects to the pipeline’s entry stage(s) according to a chosen schedule. Each *query* serves as the carrier for both the data payload and a flexible execution context (detailed in Section 3.2). This design allows the load generator to act purely as a timing signal. A data loader stage can be configured to pre-load its data and release a batch upon receiving a signal (for inference workloads), or it can perform just-in-time I/O once it receives the signal (to measure the full end-to-end training/fine-tuning pipelines).

The load pattern is determined by selecting a scheduler component in the configuration. The primary options are:

- **synchronous scheduler** that waits for the previous query to complete its full round-trip before sending a new signal. This enforces serialized execution, which is ideal for reproducing training runs.
- **Poisson-process scheduler** that send signals according to a stochastic arrival process. This models user request patterns in an inference scenario and allows researchers to study system behavior under varying load.

3.2 Execution Lifecycle

To fully understand how *McBenchface* translates a static declarative graph into a dynamic, instrumented workload, we now explain the complete lifecycle of a pipeline experiment. This process, illustrated in Figure 2, is composed of three phases. First is the orchestration and instrumentation phase managed by the RadT platform, followed by internal initialization phase which sets up the pipelines, preparing for subsequent continuous execution of the built pipelines in the last phase.

3.2.1 Phase 1: Orchestration and Instrumentation (Steps 1-4). The execution begins when user submits a configuration file (step 1). *McBenchface* parses this definition and passes this information to RadT. RadT then takes control, launching the *McBenchface* pipelines in their own isolated processes with possible collocation instrumentation (steps 2 and 3). Afterwards, RadT attaches resource utilization listeners (step 4), allowing it collect collocation-aware metrics for each of the pipeline processes using system-level utilities, such as `nvidia-smi`, `dcmi` or `iostat`. These events are collected throughout the pipelines’ lifetime and offloaded asynchronously to a RadT server together with the high-level tracing information.

3.2.2 Phase 2: Initialization (Steps 5-10). Once the pipeline process starts, it enters an initialization phase. It re-parses the configuration (step 5) to instantiate the specific *load generator* and *pipeline* objects (steps 6 and 7). Inside the pipeline, all of the *stages* are initialized, including the one-time setup in `prepare` method, and connected together using queues (step 8). Finally, *McBenchface* launches the worker threads for the *load generator* and each of the *pipeline’s stages* (steps 9 and 10).

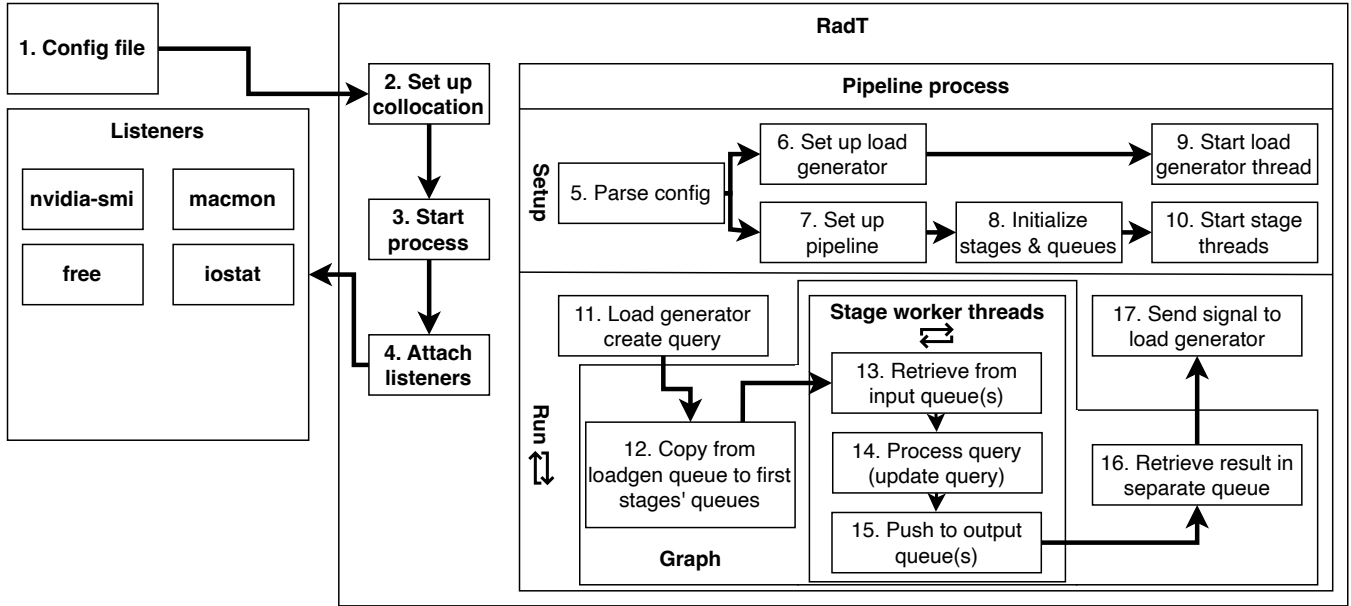


Figure 2. The execution lifecycle of a *McBenchface* pipeline. Steps 1-4 represent the orchestration phase managed by RadT. Steps 5-10 depict the initialization within the pipeline process. Steps 11-17 illustrate the execution flow over a single query.

3.2.3 Phase 3: The Query Loop and Data Flow (Steps 11-17). With the execution environment initialized and worker threads waiting, the system is ready to process data. The execution is driven by the *load generator* in a signal-based loop. The generator create a *query* object according to its schedule (step 11). The object is the primary carrier of state in *McBenchface*. It encapsulates not only the data payload but also a structured query-related context, allowing metadata to be shared across non-adjacent stages.

The *query* is pushed to the pipeline’s entry queue, which then copies these entries to the input queue(s) of first stage(s) (step 12). The stage retrieves this data (step 13) and processes the query using its run method (step 14), modifying this query in place, which is then passed to its output queue(s) (step 15) or in other words input queue of the downstream stages which then perform the steps 13 to 15 again. To handle complex topologies with fan-in pattern, such as in the case of Stage 2 of Pipeline 2 in Figure 1, *McBenchface* employs configurable polling policies (e.g., to wait for all input or accept first available or first submitted entries) to determine when a stage is ready to process data and how the data should be merged in the cases where multiple queues are used simultaneously.

Eventually, the *query* reaches the end of the pipeline (step 16) and a completion signal is sent back to the *load generator* (step 17), which is then used to trigger release of next *query* in the case of synchronous *load generator*. For scenarios requiring query-agnostic shared state (e.g., a shared model instance), stages can bypass this queue-based flow of metadata and synchronously invoke methods on other stages. These

invocations execute in the caller’s and pipeline’s thread, ensuring that access to the shared resources is not blocked.

3.3 Extensibility and Component Library

While the lifecycle described above covers standard use cases, research often requires specialized behaviors. To support these diverse requirements, *McBenchface* is designed for extensibility. Beyond creating new processing stages, as described in Section 3.1.1, researchers can implement and integrate custom behaviors for other core components, such as queue polling policies or load schedulers. Similarly to creating a new stage, this is achieved by inheriting appropriate abstract base classes and referencing the custom implementation via its Python import path in the configuration file.

This extensible architecture has enabled the development of a comprehensive ecosystem of components. *McBenchface* includes a library of pre-built stages and example pipelines, some of which will be used throughout evaluation in Sections 4 and 5. This not only provides out-of-the-box support for common ML tasks but also serves as a template for researchers to develop their own components.

The library of stages is organized by backend and functionality. For large language models, *McBenchface* provides stages for inference and finetuning that support multiple popular backends, including Hugging Face Transformers, Apple’s MLX and PyTorch’s TorchTune. For vision workloads, a suite of stages wraps the TorchVision library, enabling training and inference of popular image classification models. Beyond model execution, *McBenchface* also includes

more specialized stages for data loading, preprocessing, retrieval and control flow.

4 Evaluation

- modularity overhead - single stage pipeline vs. decomposed simple pipeline (+ other overheads?)
- ease of use (or cost of creating a new use case) - show that changing to a different model is just a change of one line in config file, how many lines does it take to define a new stage with for example vector search engine
- showcase hardware diversity, complex pipeline, and collocation - RAG - across different hardware (mac, dgx spark, big a100 and h100)
- showcase hardware diversity, complex pipeline (no collocation) - visual question answer - just on mac, but different processors on mac
- strength over mlperf - end-to-end vs not-end-to-end measurement and how it impacts hw utilization and overall conclusions - run on different hardware

Depth	Avg. Latency (ms)	Latency per Stage (μ s)
1	0.055	54.94
2	0.098	49.10
4	0.163	40.72
8	0.294	36.76
16	0.821	51.34
32	1.154	36.07
64	2.499	39.05

Table 1. *McBenchface*’s overhead across different pipeline depths. Each of the stages is passthrough only (noop).

rob *This table will change. Just putting the numbers in here for reference. These were run on macbook. Some inconsistencies might be due to radt. This depends on radt before its fixes.*

Metric	Monolithic Baseline	Choreo Pipeline	Overhead (%)
Median Latency (ms)	80.92	79.85	-1.32%
Mean Latency (ms)	82.85	81.22	-1.97%
P99 Latency (ms)	103.93	92.71	-10.79%
Std. Deviation	8.18	3.92	-52.01%

Table 2. Operational overhead comparison for EfficientNetV2-S training on Imagenette (100 batches) using *McBenchface* and its monolithic counterpart.

rob *These results do not make sense, but everything seems to be apples to apples. I think the difference might be caused by the significantly higher std. dev. I have tried this multiple times, even with 1000 batches but it did not change the results.*

rob *Waiting for a go-ahead from Ties after radt updates.*

5 Experiments

- Scalability study - run several different sizes of for example llms and compare resource utilization, shows that we can efficiently stress hardware of any size and the difference to a standardized benchmark
- collocation analysis with inference load (e.g. agent) + fine-tuning. Shows the scaling via collocation and the non-standardized benchmark aspect, where you can find the right combination of models for example.
- ablation-esque study for RAG - swap out models or databases, this helps showcase the ability to model complex workloads and swap out entire stages

Shared memory bandwidth tax. Since the shared memory also shares memory bandwidth, we need to look past the host-to-gpu and HBM memory bandwidths and look at how CPU-heavy and GPU-heavy workloads interfere together. We can use colpali to show this problem. Colpali starts by rasterizing pdfs to images (should be memory heavy), processes these images and puts them in vector database. Then, we create embeddings of questions, perform retrieval and generate answer.

Alternatively, we can run the indexing step as one workload and chat (without rag) as a secondary workload (victim).

6 Discussion

7 Conclusion

References

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009